

Design and Implementation of an Automated Code Review System Using Machine Learning and Deep Learning

A Dissertation Submitted for the Partial Fulfillment of the Requirements for SEM-3

Prepared by:

Aditya Patel

Candidate ID: AP-2202

Department of Computer Engineering

Academic Year 2026-2027

Table of Contents

Chapter	Page
1. Introduction & Motivation	3
2. Technology Stack & Literature Review	4
3. System Architecture & Methodology	5
4. Core System Implementation Details	6
5. Experimental Evaluation & Results	7
6. Conclusion & Future Directions	8

1. Introduction & Motivation

In modern software development pipelines, code review is a key safeguard for code stability, performance, and security. However, human code review can be highly time-consuming, prone to human error, and subjective. This project introduces a hybrid automated system combining traditional rule-based linting, machine learning-driven quality scoring, and advanced deep learning embeddings to evaluate and refactor source code.

Research Objectives

- Deploy a lightweight local pipeline capable of analyzing Python syntax trees and flagging code issues.
- Harness transformer-based models (CodeBERT) to extract high-dimensional semantic embeddings.
- Evaluate static and semantic features via a trained Random Forest Classifier to assign objective quality grades.
- Offer users 3 distinct, compiler-ready refactored versions of their code using Qwen2.5-Coder.

2. Technology Stack & Literature Review

Automated code review has evolved from basic regex-based linters to deep-learning models capable of understanding language semantics. This project implements a layered technology stack to achieve both syntax linting and semantic understanding.

Layer	Frameworks / Models	Core Benefit
Web Tier	Flask 3.1.3, SQLite, Bootstrap 5	Multi-user interface, history logging
Syntax Analysis	Python AST Parser, Pylint	Indentations, function counts, logic bugs
Semantic Embeddings	CodeBERT (microsoft/codebert-base)	Extract semantic vector representation
ML Classification	Random Forest Classifier (scikit-learn)	Quality prediction (Excellent/Good/Poor)
AI Refactoring	Qwen2.5-Coder-0.5B-Instruct	Zero-shot PEP 8 formatting, optimization

3. System Architecture & Methodology

The core logic utilizes a hybrid scoring and classification engine. The score calculator applies custom deductions based on rule severities:

Formula 1: Score Calculator Deductions

$$\text{Quality Score} = \max(0, 100 - \text{sum}(\text{Deduction Weight} * \text{Issue Count}))$$

Semantic similarity is computed by encoding code strings into 768-dimensional vector spaces using CodeBERT, and calculating the cosine similarity against an optimized reference database:

Formula 2: Cosine Similarity Calculations

$$\text{Cosine Similarity} = (\mathbf{A} \cdot \mathbf{B}) / (||\mathbf{A}|| * ||\mathbf{B}||)$$

Deduction Weightings Matrix:

Severity	Pylint Key	Deduction Weight
Fatal	F	20.0
Error	E	15.0
Warning	W	10.0
Refactor	R	5.0
Convention	C	2.0

4. Core System Implementation Details

Below are excerpts of the core modules orchestrating the machine learning classification, model inference, and refactoring pipelines.

Qwen-Coder Refactoring Assistant

```
class CodeImprover:
    def improve(self, source_code, issues=None):
        clean_sys = "You are a PEP 8 styling formatter..."
        clean_code = self._safe_generate(source_code, clean_sys)
        bp_sys = "You are a Python best practices expert..."
        best_practice = self._safe_generate(source_code, bp_sys)
        opt_sys = "You are a Python performance tuning compiler..."
        optimized_code = self._safe_generate(source_code, opt_sys)
        return { "clean_code": clean_code, "best_practice": best_practice,
"optimized_code": optimized_code }
```

Feature Extractor Integration

```
class MLFeatureExtractor:
    def extract(self, file_path):
        ast_data = self.ast_analyzer.analyze(file_path)
        pylint_data = self.pylint_analyzer.analyze(file_path)
        semantic_data = self.semantic_analyzer.analyze(file_path)
        return [ ast_data['functions'], ast_data['classes'], pylint_data['issue_count'],
... ]
```

5. Experimental Evaluation & Results

To validate system accuracy, we compiled a feature dataset from the CodeSearchNet database and evaluated the models.

Random Forest Classification Metrics

Quality Label	Precision	Recall	F1-Score	Support
Excellent	0.83	0.78	0.80	12
Good	0.75	0.70	0.72	10
Average	0.70	0.80	0.75	15
Poor	0.71	0.71	0.71	7
Average / Total	0.75	0.75	0.75	44

Key Findings:

- **Rule Deductions vs ML Grade:** Static logic is useful for styling errors, but the ML classifier successfully detects architectural structural defects (redefining variable scope, complexity) not caught by simple subtraction logic.
- **Qwen-Coder Speed:** The 0.5B instruction-tuned model completes code refactoring in ~12 seconds per version on standard single-core CPUs.

6. Conclusion & Future Directions

This automated system bridges the gap between rule-based linters and generative models by combining AST statistics, deep learning vector similarities, and target-based generative refactorings into a unified Flask platform. The 75% classifier accuracy and local Qwen CPU generation demonstrate that automated reviews can be run locally on lightweight hardware.

Future Scope

- **Multi-language Support:** Adapt tokenizers and compilers to parse C++, Java, and Javascript AST modules.
- **Automated PR Hooks:** Build webhook triggers for GitHub Actions and GitLab CI/CD pipelines to review pull requests instantly.
- **Local LLM Quantization:** Implement 4-bit GGUF quantization models to decrease memory overhead to <1GB.